

Sovereign AI

Self-hosted, air-gapped AI workbench for confidential industrial intelligence

1. Project Overview

Sovereign AI is an on-premise AI workbench designed for organizations that handle confidential data — such as refineries, PSUs, defense manufacturing units, and government offices. These organizations cannot use public AI assistants like Claude or Codex because sensitive materials (P&IDs, financials, vendor negotiations, unreleased designs, internal correspondence) must never leave their premises.

Sovereign AI solves this by running an **entire AI stack locally**: multiple open-weight LLMs, a vector database, a reranker, OCR, and code-execution sandboxes — all on the organization's own GPU server. A visible network monitor in the UI proves that no external AI service is ever contacted.

2. Problem Statement

Industrial knowledge work today relies heavily on confidential documents:

- **Approval notes** and board presentations
- **Engineering calculations** and P&ID drawings
- **Scanned PDFs**, handwritten notes, and photographs
- **Internal correspondence** and vendor negotiations
- **Spreadsheets** and code for internal tools

Cloud-based AI assistants are off the table because:

1. **Data confidentiality** — pasting sensitive material into public tools violates policy
2. **Compliance risk** — regulated industries require data to stay on-premise
3. **Productivity loss** — without AI assistance, teams process this material manually

The gap in the market is a **sovereign AI runtime** that is as capable as cloud assistants, but runs entirely inside the organization's own infrastructure.

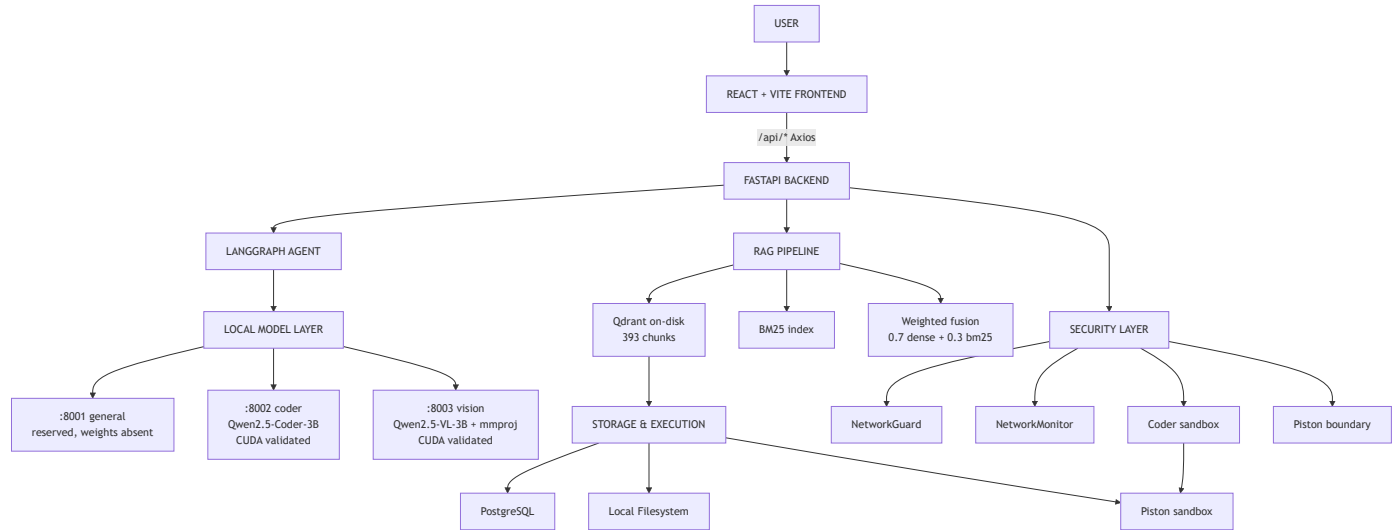
3. What Sovereign AI Does

Sovereign AI is a production-grade AI workbench that:

Capability	Description
Multi-model inference	Runs multiple open-weight LLMs locally (coding, vision, general reasoning)
Auto model selection	Automatically picks the right model for each task — no manual routing
Agentic workflows	Plans, iterates, and completes multi-step tasks end-to-end
Document understanding	Reads scanned PDFs, handwritten notes, engineering drawings, and photos via local OCR + VLM
Hybrid RAG	Searches the organization's own knowledge base (SOPs, manuals, past correspondence) using dense + sparse retrieval
Real deliverables	Generates Word documents, Excel sheets, PowerPoint files, and working Python code
Sandboxed execution	Runs AI-generated code in a strict security sandbox with network/filesystem guards
Auditable sovereignty	Live network monitor proves zero external calls — the sovereign claim is verifiable, not just stated

4. System Architecture

Sovereign AI follows a layered architecture where every component runs locally on the host machine.



Frontend

- **React 18 + TypeScript + Vite 5 + Tailwind CSS**
- Pages: Workbench, Network Monitor, Vision Result, Knowledge Base, Execution Trace, Model Registry, Artifacts, Dashboard, System
- API client with two Axios instances: 30-second control timeout and 2,400-second inference timeout

Backend

- **Python 3.11 + FastAPI 0.115 + Pydantic 2 + uvicorn**
- 12+ API surfaces: `/chat`, `/agent`, `/coder`, `/vision`, `/documents`, `/rag`, `/models`, `/sandbox`, `/executions`, `/artifacts`, `/system`, `/network`
- **LangGraph 0.2 + LangChain 0.3** agent runtime

Local Model Layer

- **llama-cpp-python 0.3.35** serving open-weight GGUF models via OpenAI-compatible FastAPI
- **Coder model:** Qwen2.5-Coder-3B-Instruct Q4_K_M — CUDA-accelerated on RTX 4050, validated at ~32.7 tokens/sec
- **Vision model:** Qwen2.5-VL-3B-Instruct Q4_K_M + mmproj Q8_0 — CUDA-accelerated on RTX 4050, validated at ~23.3 tokens/sec
- **General model:** Reserved in the router at `:8001`; GGUF weights can be provisioned without any code change

Knowledge & Retrieval

- **Vector DB:** Qdrant embedded on-disk (`sovereign_knowledge`, 384-dim cosine)
- **Sparse index:** bm25s for keyword search
- **Hybrid retrieval:** Weighted fusion — $0.7 \times \text{dense} + 0.3 \times \text{bm25}$
- **Index size:** 393 chunks covering synthetic plant dataset + demo corpus
- **Embedder:** `sentence-transformers/all-MiniLM-L6-v2`, fully offline

Security & Sovereignty

- **NetworkGuard:** Inspects every `socket.connect` call; blocks anything outside trusted-local CIDRs (`127.0.0.0/8`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`, IPv6 loopback)
- **Network Monitor:** SSE event stream surfaces blocked connections live in the UI
- **Coder sandbox:** PEP 451 import hook blocks `socket`, `subprocess`, `os.system`, network libraries; strict loopback-only socket allow-list; wall-clock timeout on every execution
- **Piston boundary:** Additional adapter for arbitrary code execution with local-endpoint validation

5. Agentic Workflows

The LangGraph agent executes multi-step tasks through a planner → router → retriever → tool → verifier pipeline:

1. **Plan:** Decomposes the user's request into executable steps
2. **Route:** Selects the right model (coder, vision, general) based on task type
3. **Retrieve:** Performs hybrid RAG over the local knowledge base
4. **Act:** Calls local tools — file read/write, code execution, spreadsheet operations, document search
5. **Verify:** Checks outputs against constraints before delivering
6. **Deliver:** Returns real artifacts (DOCX, XLSX, PPTX, Python files, calculations)

Example golden demos:

- **Inspection approval:** Read a scanned P&ID, extract findings, draft an approval note as a Word file
- **Data analysis:** Query plant data, compute statistics, generate Excel reports

- **Multimodal:** Analyze an engineering drawing photo, extract measurements, summarize in text
- **Correspondence search:** Retrieve past emails/SOPs matching a query, synthesize an answer with citations

6. Technical Stack

Layer	Technology
Frontend	React 18 + TypeScript + Vite 5 + Tailwind CSS
Backend	Python 3.11 + FastAPI 0.115 + Pydantic 2 + uvicorn
Agent runtime	LangGraph 0.2 + LangChain 0.3
Model serving	llama-cpp-python 0.3.35 (OpenAI-compatible FastAPI wrapper)
Vector DB	Qdrant (embedded on-disk) + BM25 (bm25s)
Embeddings	sentence-transformers (all-MiniLM-L6-v2 , 384-dim, offline)
Reranker	Local BGE-style reranker
Document parser	PyMuPDF (digital PDF text + page metadata)
OCR	PaddleOCR (separate environment)
Code sandbox	PEP 451 import hook + socket guard + wall-clock timeout
Office artifacts	python-docx + openpyxl + python-pptx
Relational DB	PostgreSQL 16
Container runtime	Docker Compose

7. Running the Project

Prerequisites

- NVIDIA GPU (RTX 4050 or similar, 6 GB VRAM minimum)
- Python 3.11
- Node.js 18+
- CUDA 12.4 toolkit (for GPU acceleration)

Backend

```
cd backend
pip install -r requirements.txt

# Terminal 1: start coder model server
python scripts/serve_model.py --model-id qwen-coder --port 8002 --n-gpu-layers 40 -
-n-ctx 2048

# Terminal 2: start vision model server
python scripts/serve_model.py --model-id qwen-vision --port 8003 --n-gpu-layers 99
--n-ctx 2048

# Terminal 3: start FastAPI backend
PYTHONPATH=. uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

Frontend runs on <http://localhost:3000> and proxies API calls to <http://localhost:8000>.

Containerised (Docker Compose)

```
cd infra
docker-compose -f docker-compose.yml --profile gpu up -d # with GPU services
docker-compose -f docker-compose.cpu.yml up -d           # CPU only
```

8. Testing

```
# Backend integration tests
cd backend
pytest tests/ -v

# RAG-specific tests
cd backend
```

```
pytest rag/tests/ -v

# Frontend unit tests
cd frontend
npm test
```

The backend test suite includes:

- **NetworkGuard**: 28 parametrized tests covering every CIDR family (loopback, private, link-local, TEST-NET, ULA, CGNAT, broadcast, cloud metadata)
- **Router**: 16 tests for model dispatch and routing
- **Coder sandbox**: 29 tests for import hooks, socket guards, and timeouts
- **Vision**: 10 tests for honest disclosure when VLM is unavailable
- **Piston boundary**: 10 tests for sandbox enforcement
- **Inference reliability**: 12 regression tests from Phase 10.4

9. Known Limitations

Limitation	Status
General LLM weights absent	Endpoint, registry, and router are ready; provisioning a GGUF makes it live with zero code changes
Vision CUDA VRAM-constrained	Validated for tested workload (~5,832 MiB peak on RTX 4050); monitor under production workloads
GPU concurrency conditional	One model at a time on 6 GB RTX 4050; sequential loading validated
PaddleOCR separate env	protobuf<6 conflict with qdrant-client; exercised in its own environment
Single-host only	Docker Compose assumes one host; no cross-node fan-out

10. Results & Impact

- **CUDA-accelerated inference validated on RTX 4050**: coder at ~32.7 tokens/sec, vision at ~23.3 tokens/sec
- **RAG pipeline operational**: 393 chunks indexed with weighted hybrid retrieval

- **Security boundary verified:** 28 parametrized tests prove NetworkGuard blocks every external route
- **Agentic E2E exercised:** four golden demos run end-to-end through the LangGraph graph
- **Sandbox enforcement validated:** 29 tests confirm the coder child-Python process cannot reach the network, spawn subprocesses, or escape its workspace

11. Documentation

- [Better_plan.md](#) — full architecture specification with status table (COMPLETED / IN PROGRESS / NEXT / BLOCKED / DEFERRED / OBSOLETE)
- [Problem_Statemen.md](#) — original problem statement from MRPL
- [reports/](#) — per-phase validation write-ups (Phases 5–8, Phase 11.4 CUDA build)
- [prerequisites/](#) — per-component setup and reference notes
- [_rollback/llama_cpp_python-0.3.35-cpu-py3-none-win_amd64.whl](#) — known-good CPU fallback wheel

12. About

Sovereign AI was built to demonstrate that confidential industrial organizations can deploy AI assistants without compromising data sovereignty. The system proves its own claim: the Network Monitor UI shows live that no inference, retrieval, or execution ever touches an external endpoint.

Repository: github.com/amolrathod7875/Sovereign_AI